

BUILD AND ORCHESTRATE MULTI-AGENT WORKFLOWS

Using Microsoft Foundry Agent Service

AZURE FEST 2026

Alpa Buddhabhatti • Microsoft MVP • Microsoft Certified Trainer

About me

Alpa Buddhahatti

Lead Data & AI Engineer / Architect
Microsoft MVP — Data Platform
Microsoft Certified Trainer (MCT)

Focus: Microsoft Fabric, Azure Data & AI,
Microsoft Foundry

 @AlpaB7

 @alpabuddhabhatti

 @alpaBuddhabhatti

 @alpabuddhabhatti

 @meetalpa

What I want you to leave with

A simple way to think about multi-agent orchestration:

- Small specialist agents
- Explicit workflow control
- Human approval where it matters
- Identity, traces and evaluation around the workflow

Facts → Rules → Resolution → Control → Action

Agenda

One orchestration idea — from visual workflow to code-first Agent Framework

01

Foundry in 2 minutes

Build, run, observe and govern agents.

02

Agents → Multi-Agent

Why specialists can improve separation of responsibilities.

03

Sequential pattern

Our starting point: a clear, dependent business process.

04

Retail scenario + data

Understand ORD-1048 before touching the demo.

05

Three demos

Portal → Toolkit in VS Code → Agent Framework.

06

Production next steps

Tools/APIs, identity, evaluation, observability, promotion.

Microsoft Foundry

Session view: Build → Orchestrate → Run → Observe → Govern



No-code / low-code when enough → code-first when deeper control is needed

Today we deliberately move across that spectrum using the same orchestration idea.

From one agent to a team of specialists

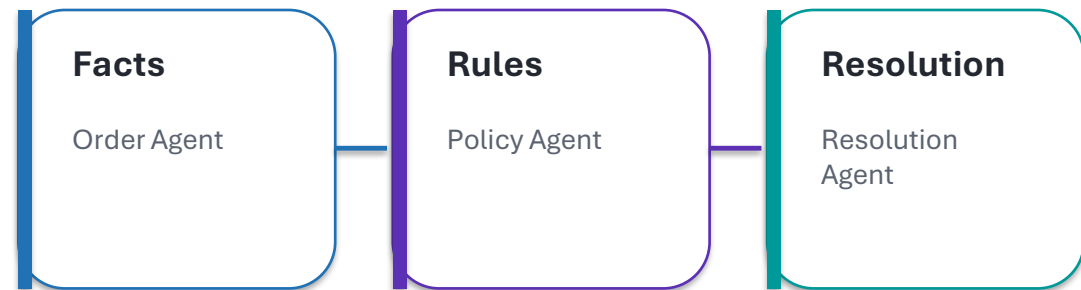
Multi-agent is useful when responsibilities, access or controls need clearer boundaries

Single Agent

- One prompt
- Many tools
- Many responsibilities
- Broad context
- Harder to isolate failures

Multi-Agent

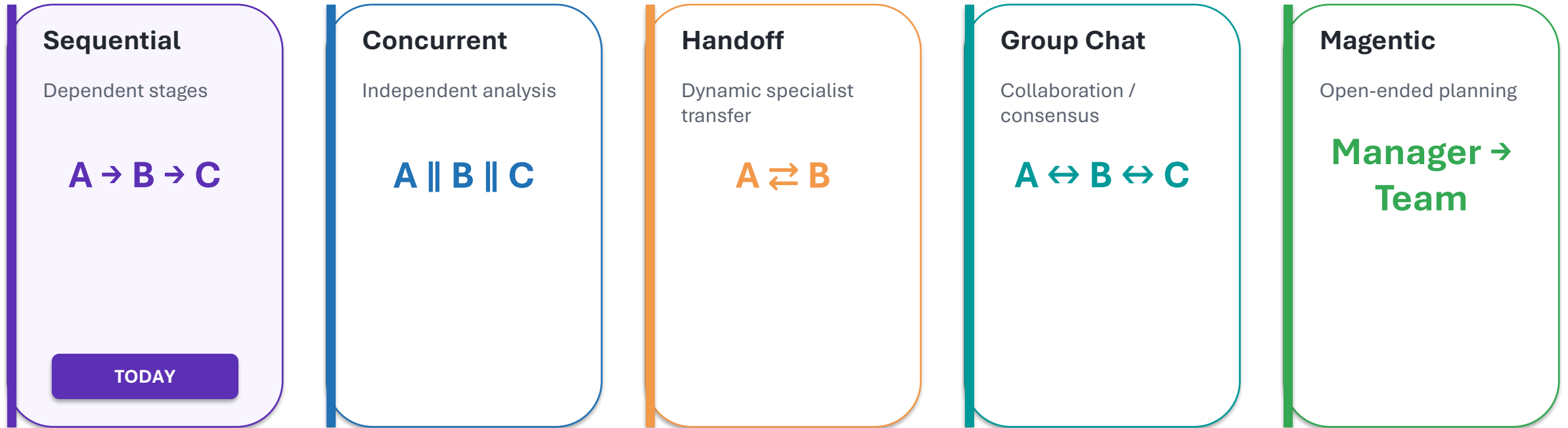
Small specialists + an explicit coordination model



Specialise the intelligence. Keep business control explicit.

Multi-agent orchestration patterns

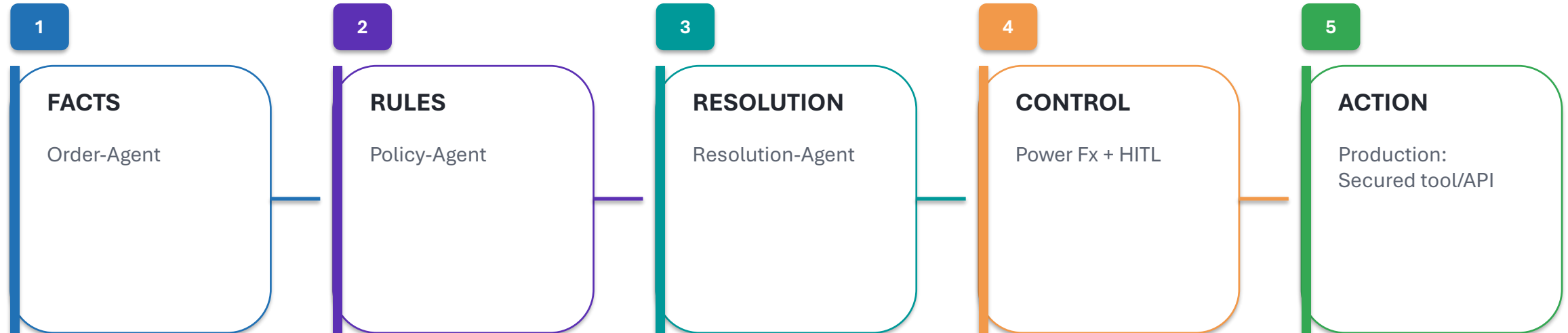
Microsoft documents five core patterns. Today we focus on Sequential.



Start with the lowest orchestration complexity that reliably solves the business problem.

Why start with Sequential?

Our business process already has a clear order

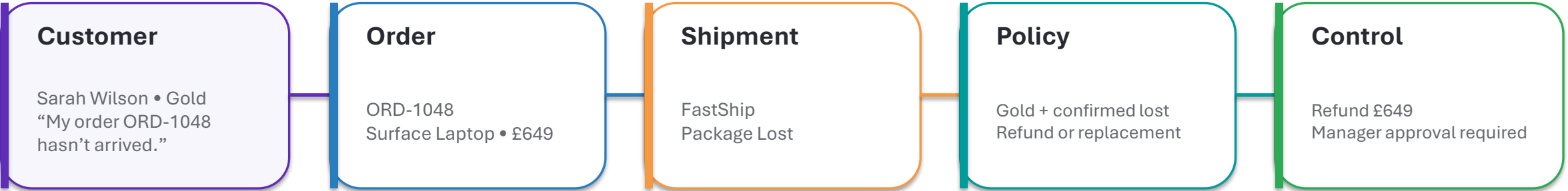


Deterministic business rules should remain deterministic.

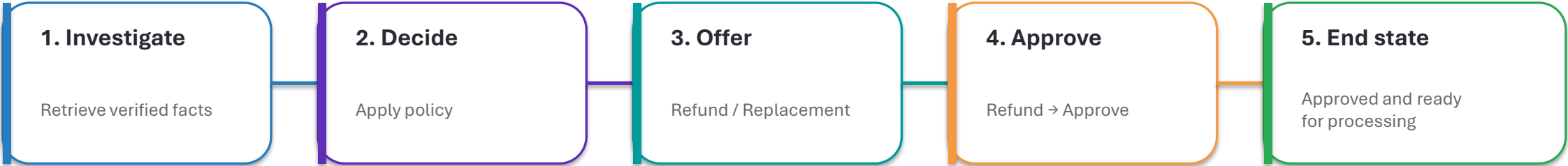
“The LLM recommends; the workflow controls.”

Scenario: Retail Order Resolution

A missing order becomes a useful end-to-end multi-agent business process



GOLDEN DEMO PATH

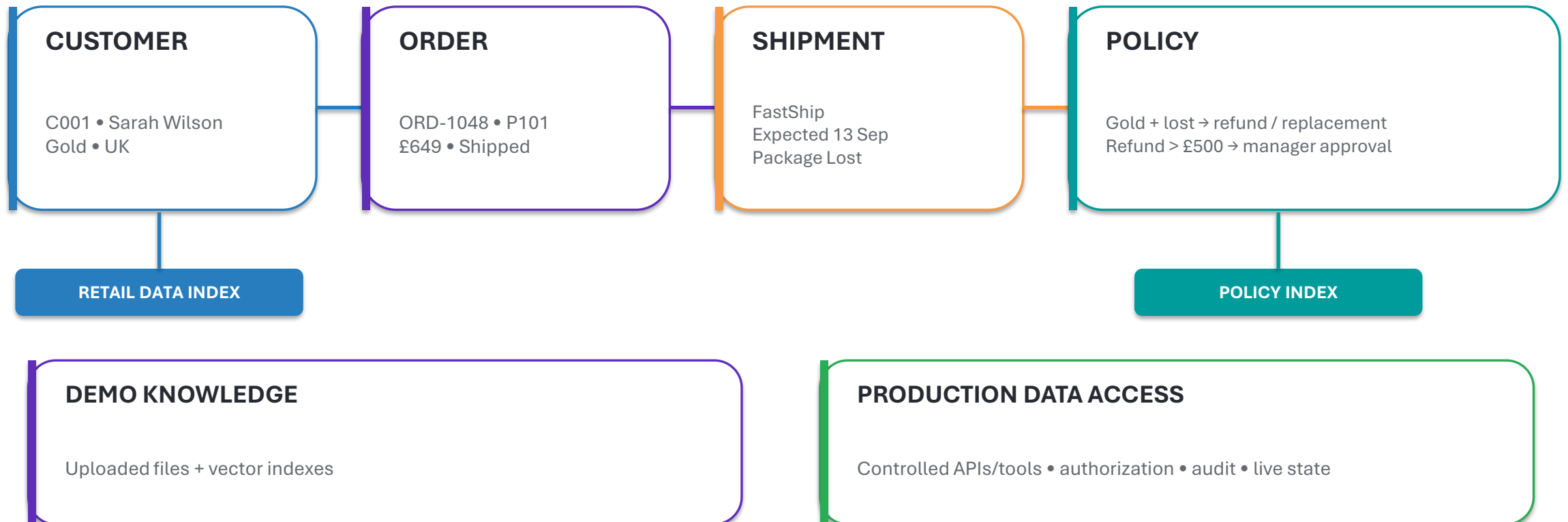


Nofinancial action is executed.

Facts → Rules → Resolution → Control → Action

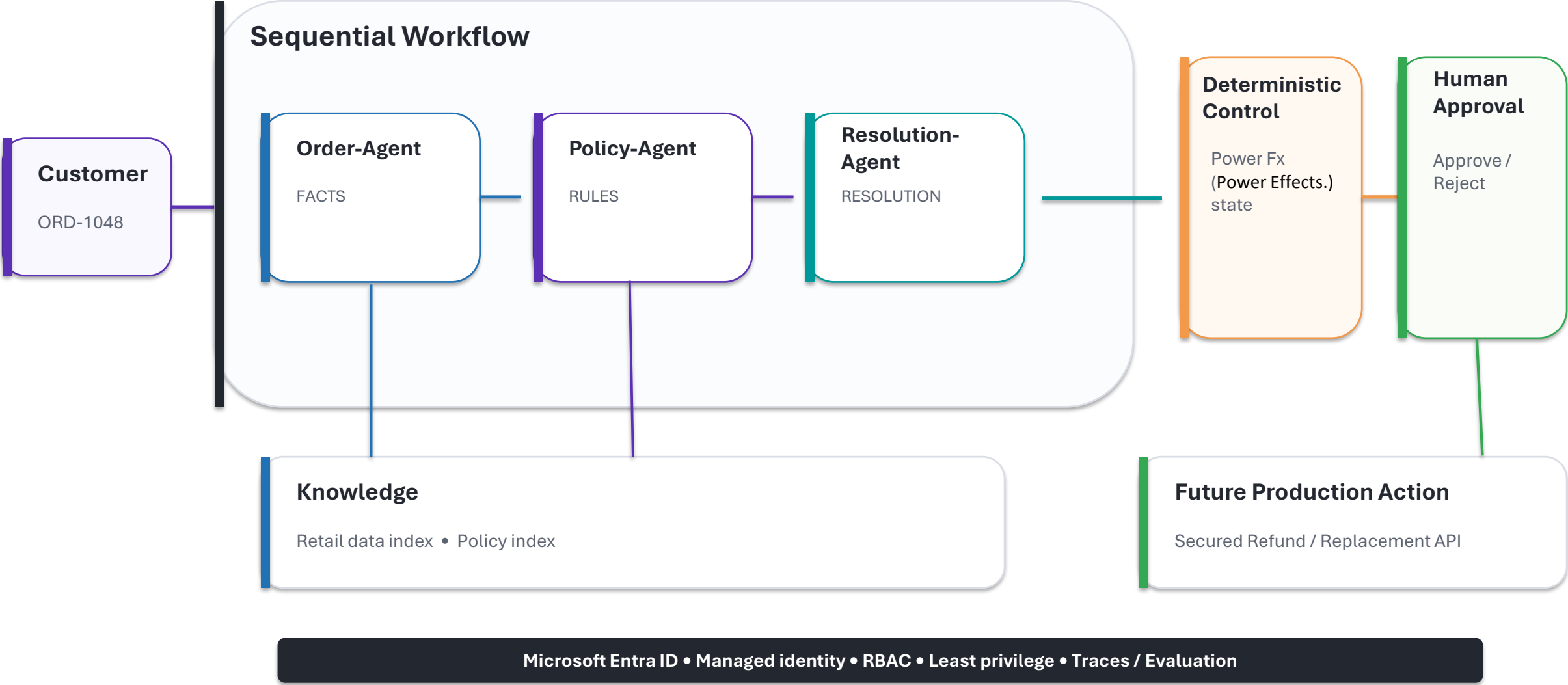
Demo data and knowledge

Self-contained for the conference; production operational state should normally come from controlled APIs/tools



- Knowledge helps the agent know something.
 - A tool helps the agent do something.

Demo architecture



Demo

DEMO 1

Foundry Portal

Retail workflow

- Agents
- knowledge
- visual workflow
- Power Fx
- HITL

DEMO 2

Foundry Toolkit

Same v23 workflow in VS Code

- Visual graph
- YAML
- test
- developer lifecycle

DEMO 3

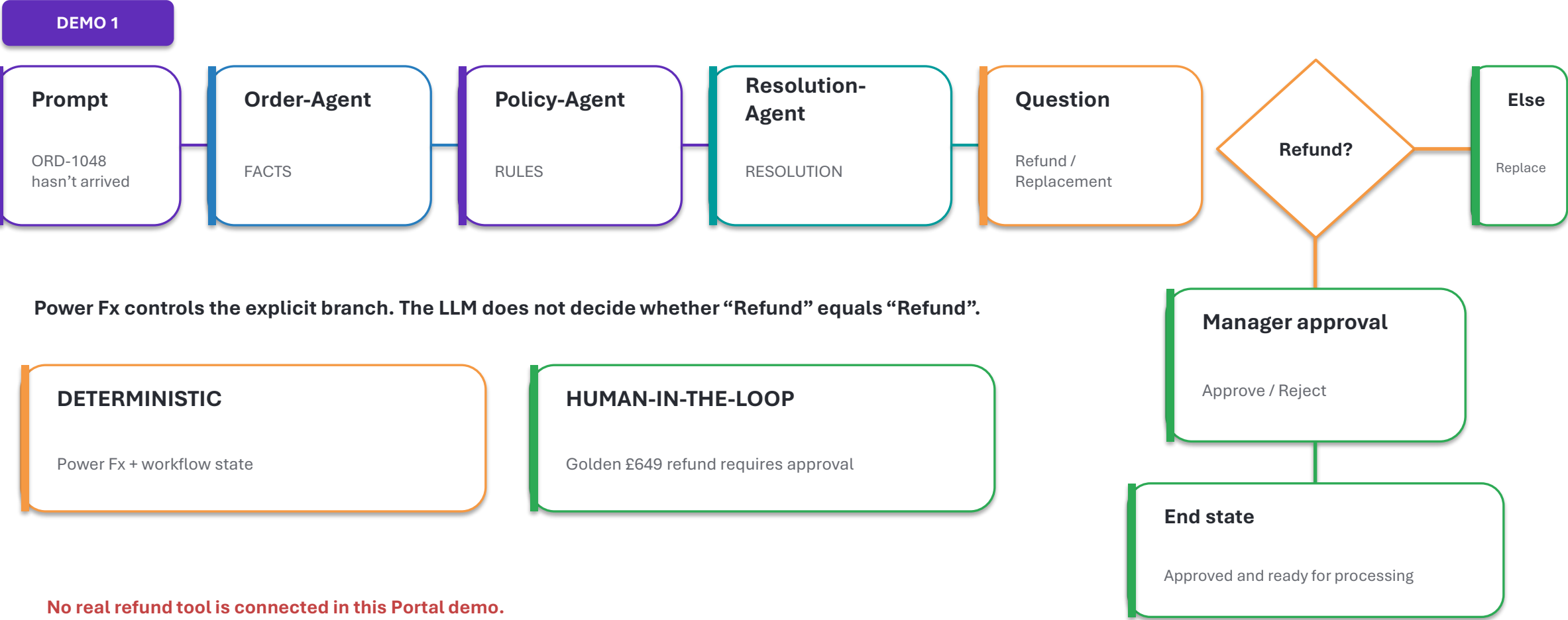
Agent Framework

WorkflowBuilder → Foundry Hosted Agent

- Code
- Agent Inspector
- deployment
- hosted trace

Demo 1 — Foundry Portal

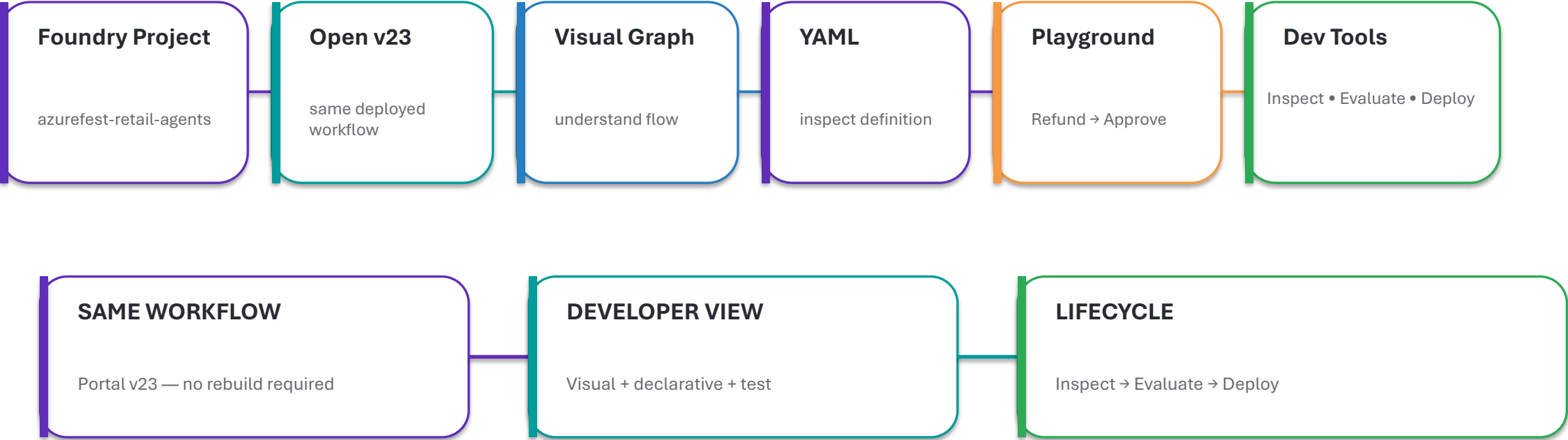
The business story is more important than the clicks



Demo 2 — Foundry Toolkit in VS Code

The same workflow moves into the developer lifecycle

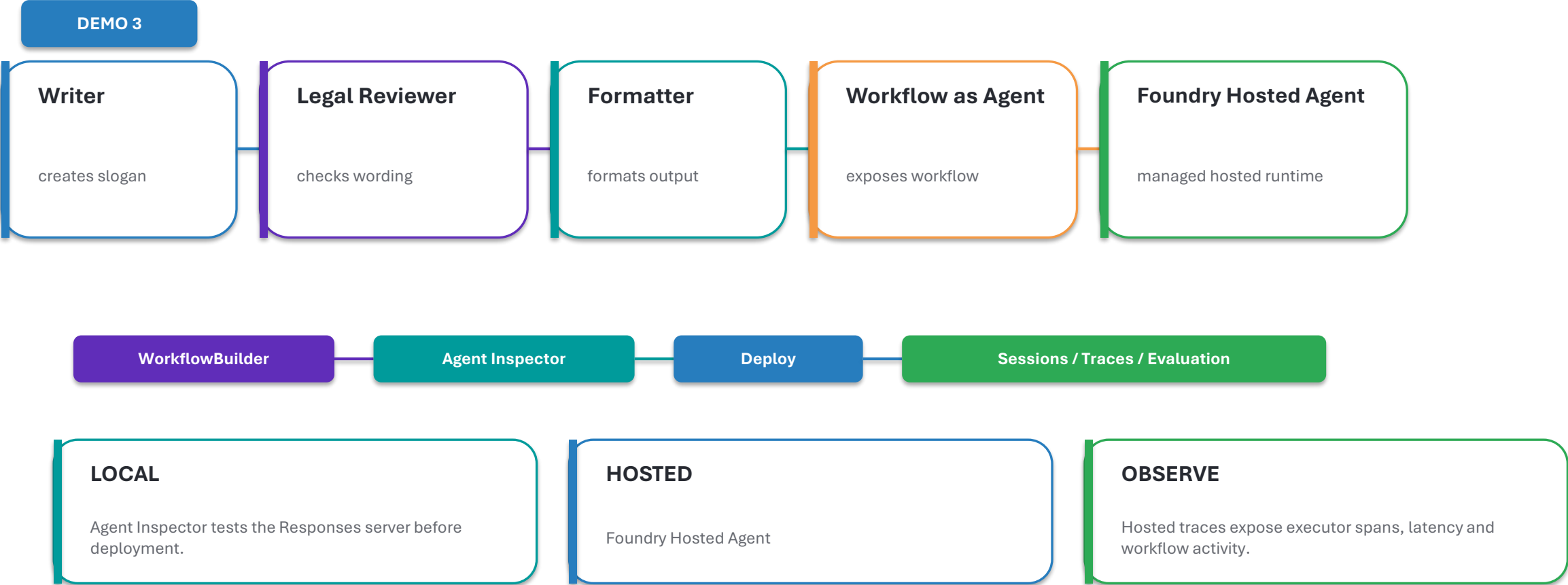
DEMO 2



Portal shows the orchestration. Toolkit brings it into the developer lifecycle.

Demo 3 — Microsoft Agent Framework

A real WorkflowBuilder sample: local test → hosted deployment → trace



No API key embedded in code — identity-based authentication + least privilege.

If time permits

Copilot-assisted tooling lowers the entry barrier; engineering controls still matter



COPILOT CAN HELP

Draft boilerplate • explain YAML/code • suggest tests • troubleshoot

ENGINEERING STILL CONTROLS

Variables • deterministic conditions • approvals • authorization • audit

Copilot lowers the entry barrier — it does not replace business rules, identity, testing or governance.

From demo to production

Keep agentic reasoning flexible; make critical controls explicit and observable

Live data

Replace static files with controlled APIs/tools

Real actions

Refund / replacement tools after authorization

Identity

Entra ID + least-privilege RBAC

Evaluation

Dataset-based task, quality and safety checks

Observability

Traces, tokens, latency, failures and cost

Delivery

DEV → TEST → PROD with environment-specific config

Key takeaways

A simple mental model for multi-agent business workflows

1

Specialise

Give each agent one clear responsibility.

2

Orchestrate

Choose the simplest pattern that fits.

3

Control

Use deterministic logic for deterministic rules.

4

Secure

Identity + least privilege; no credentials in prompts.

5

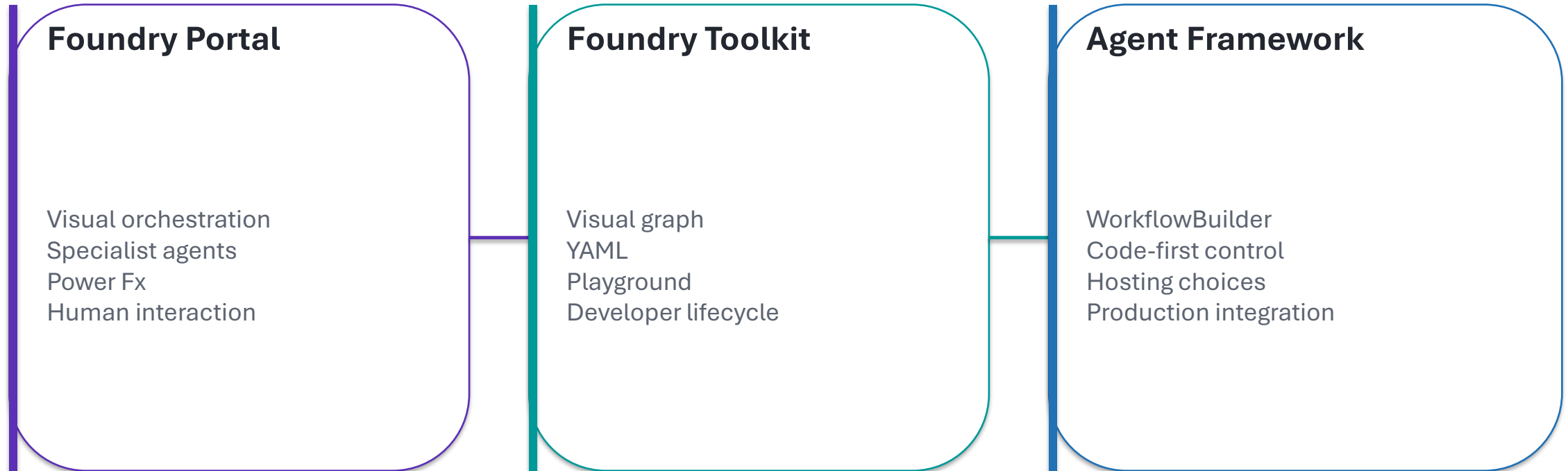
Observe

Trace, evaluate and improve before scaling.

Facts → Rules → Resolution → Control → Action

Portal → Toolkit → Agent Framework

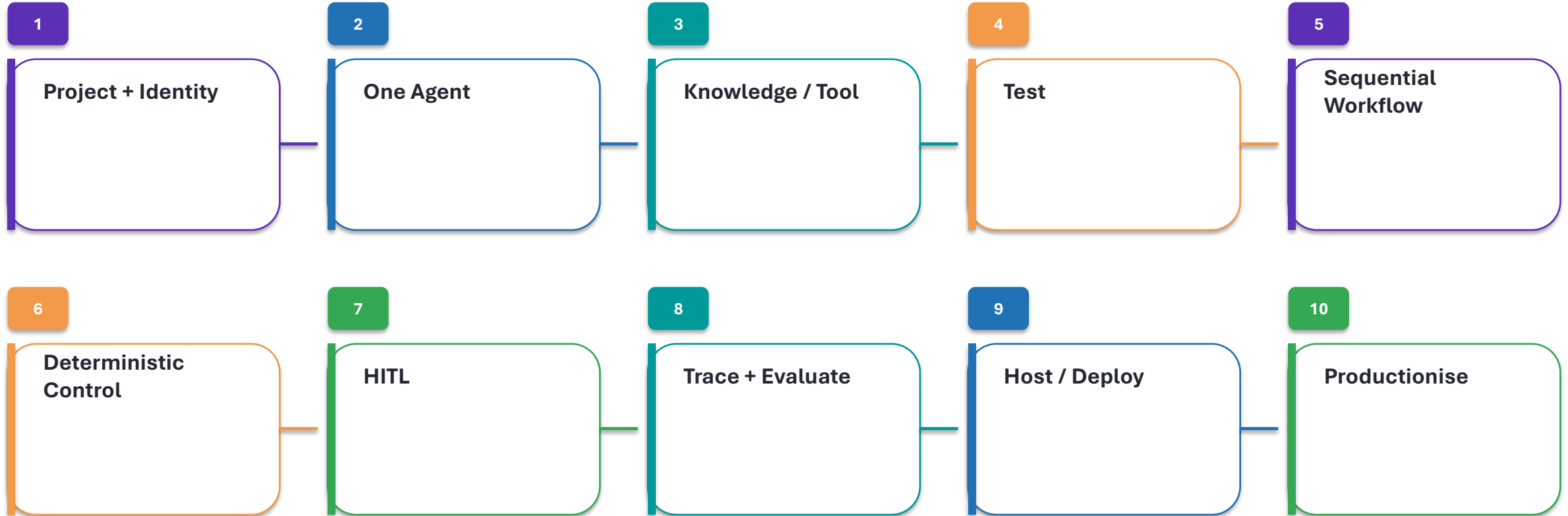
Use the Portal to learn the orchestration visually; use the forward developer path for deeper control



The orchestration concepts stay the same even as the implementation experience changes.

If I were starting my Foundry journey today...

Start simple. Prove each layer before adding the next.



Do not start by building five agents. Start with one agent that works — then separate responsibilities only when there is a reason.

Thank you

Build small specialists. Orchestrate deliberately. Keep control explicit.

Questions?

Alpa Buddhahatti • Microsoft MVP • Microsoft Certified Trainer

Azure Fest 2026

 @AlpaB7

 @alpabuddhabhatti

 @alpaBuddhabhatti

 @alpabuddhabhatti

 @meetalpa